第 0 章　計算環境を準備する

本書では、Python と Jupyter Notebook を使って数値計算を行います。本章では、第 1 章以降の Notebook を実行するために必要な計算環境を準備し、Jupyter Notebook の基本的な使い方を確認します。

Python の文法、NumPy、Matplotlib の一般的な使い方をここで解説することはしません。必要なコードは各章の物理・数値計算とともに示します。本章の目的は、**配布された Notebook を自分の計算機で開き、上から実行できる状態にすること**です。

0.1 配布物と必要なソフトウェアを確認する

0.1.1 本書のファイルを確認する

配布物には、本文のほかに完成版と学生版の Notebook があります。章番号 `XX` に対して、基本的な役割は表0.1のとおりです。

表 0.1　配布物の主なファイル。`XX` は `00` から `07` までの章番号です。

ファイル	役割
`notebook/complete/chapXX_complete.ipynb`	完成したコードと標準結果を確認する Notebook
`notebook/student/chapXX_student.ipynb`	TODO を埋めながら実装する Notebook
`code/requirements.txt`	本書の Notebook を実行するために必要な Python パッケージ

Notebook は章ごとに独立して実行できるようにしています。例えば第 4 章を実行するために、第 1–3 章の Notebook を先に実行しておく必要はありません。

0.1.2 必要なソフトウェアを確認する

本書の標準計算には GPU やスーパーコンピュータは必要ありません。一般的なノート PC で実行できる範囲の計算を扱います。必要なソフトウェアは次のとおりです。

- Python 3（3.10 以降を推奨）
- NumPy
- Matplotlib
- JupyterLab または Jupyter Notebook

すでに授業や研究室で指定された Python 環境がある場合は、その環境を使って構いません。新しく環境を作る場合は、次節の手順で配布物に含まれる `code/requirements.txt` を使います。

0.2 Python 環境を準備する

0.2.1 Python が使えるか確認する

まず端末を開き、Python が利用できるか確認します。macOS や Linux では、環境によって `python` または `python3` を使います。

端末で実行

```
python --version
```

版番号が表示されれば Python を実行できます。コマンドが見つからない場合は、授業で指定された方法で Python を導入してください。

0.2.2 仮想環境を作る

他の授業や研究で使う Python 環境と混ざらないよう、必要に応じて仮想環境を作ります。配布物の最上位フォルダで、例えば次を実行します。[7]

端末で実行

```
python -m venv .venv
```

仮想環境を有効にするコマンドは使用するシェルによって異なります。

環境	有効化するコマンド
macOS / Linux（bash, zsh）	`source .venv/bin/activate`
macOS / Linux（tcsh）	`source .venv/bin/activate.csh`
Windows PowerShell	`.venv\Scripts\Activate.ps1`

仮想環境を有効にした後、必要なパッケージをインストールします。

端末で実行

```
python -m pip install -r code/requirements.txt
```

すでに必要なパッケージが導入された環境を使う場合は、この操作を毎回行う必要はありません。

0.3 Jupyter Notebook を使う

0.3.1 Jupyter を起動する

配布物の最上位フォルダで、JupyterLab を使う場合は次を実行します。[8]

端末で実行

```
jupyter lab
```

Jupyter Notebook を使う場合は `jupyter notebook` とします。ブラウザが開いたら、`notebook/complete/` または `notebook/student/` から使う章の Notebook を開きます。

0.3.2 セルを実行する

Notebook には、文章を表示する Markdown セルと、Python を実行する Code セルがあります。Code セルを選び、通常は `Shift+Enter` で実行します。

Notebook は、セルを**実行した順序**に変数や関数の状態を保持します。そのため、標準結果を再現するときは、上から順にセルを実行してください。途中のセルだけを先に実行すると、画面に見えているコードと実際の計算状態が一致しないことがあります。

0.3.3 最初から計算し直す

結果が標準例と合わないときや、パラメータを変更して計算をやり直すときは、カーネルを再起動してから全セルを上から実行します。JupyterLab や Jupyter Notebook には、カーネルの再起動と全セル実行に相当する操作があります。メニュー名は版によって多少異なります。[8]

エラーが出た場合は、まずエラー表示の最後の数行を確認してください。`NameError`、`ValueError`、`ModuleNotFoundError` など、エラーの種類と原因を示す情報が表示されます。パッケージが見つからない場合は、Jupyter が使用している Python 環境と、パッケージをインストールした環境が同じか確認します。

0.4 完成版と学生版 Notebook を使い分ける

本書では、完成版と学生版の Notebook を次のように使い分けます。

完成版 Notebook 本文で説明した標準計算を実行し、コードと結果を確認します。

学生版 Notebook 一部のコードが TODO になっています。本文を参照して実装し、完成版と比較します。

最初は、次の順序で進めることを想定しています。

1. 本文で問題設定、数式、計算手順を確認します。
2. 完成版 Notebook を上から実行し、標準結果を再現します。

3. 学生版 Notebook の TODO に取り組みます。
4. 各章の演習では、指定された条件を変更して結果を比較します。

Notebook を編集するときは、元の配布ファイルを残して別名で保存しておくと、標準設定へ戻りやすくなります。

0.5 最初に動作を確認する

最後に、完成版の第 0 章 Notebook を開き、次のセルを実行します。

```
import sys
import numpy as np
import matplotlib

print("Python:", sys.version.split()[0])
print("NumPy:", np.__version__)
print("Matplotlib:", matplotlib.__version__)
```

Python、NumPy、Matplotlib の版番号が表示され、エラーが出なければ準備は完了です。Jupyter 上でこのセルを実行できたことを確認したら、第 1 章へ進んでください。